

PREPRINT / EXPLORATORY STUDY

Subagent coordination: SAGE-CREH and skill selection

Cristhian Egoavil / 13 September 2026 / v2.0

Abstract

We propose SAGE-CREH, a fifth architecture combining skill selection, a direct path, independent solvers and conditional adjudication. A focused review of twelve arXiv papers informs the design. After 20 separate development runs, 216 live runs used the same model and twelve original tasks, two repetitions and nine conditions: four architectures, a single agent, SAGE and three ablation controls. Replication consumed 707,090 tokens in 510 calls. SAGE achieved 23/24 strict successes and 23/24 after normalizing only the SQL envelope, averaging 1,765 tokens and 7.34 seconds per task. Explicit controls distinguish coordination, instruction loading and output format. The sample is small, synthetic and known during design; this is a development-informed replication, not evidence of universal superiority or blind generalization.

Keywords: subagents; adaptive delegation; skills; tokens; latency; paired evaluation.

Code and data: <https://github.com/CristhianRudolf7/sage-creh>

1 Research question

The unit of design is not the model but the organization of its calls: who solves, who checks, when to delegate, and which instructions each role receives. Holding the model fixed, we ask whether a fifth, adaptive architecture can retain the completion achieved by the four original architectures with less coordination. Three objectives remain distinct: completing the whole task, reducing aggregate tokens, and shortening user-perceived waiting time. None substitutes for another.

Version 1 compared sequential, parallel, hierarchical and reflexive flows with a single agent and a skill-loading ablation. Version 2 implements SAGE-CREH, evaluates two development policies and reruns every control. The contribution is this bounded implementation with reproducible evidence, not universal superiority or the invention of selective delegation.

2 Documentary foundation: nine original references

The review includes nine papers first submitted to arXiv between February and September 2026. Submission dates were checked separately from revision dates; exact consulted versions appear in the references. Selection targeted coordination, context transfer, skill assignment and measurable efficiency. This is a focused technical review, not an exhaustive systematic review. Other studies' scores are not combined with the experiment below.

2.1 Execution topology and authority

Kulkarni and Kulkarni compare sequential, parallel, supervisor–worker and reflexive workflows for financial document extraction. Their descriptions motivate the four pattern families studied here. The domain and January 2025 model/pricing setup constrain transfer to a personal assistant; we do not reproduce their configurations or reported performance.¹

DESBench examines centralized, hierarchical, heterarchical and holonic coordination in industrial scheduling. Its implemented heterarchy uses mediated bidding. It expands the design question from graph shape to who may allocate work, reject commitments and resolve conflicts; its industrial simulator is a different evaluation environment.²

ParaGUI dispatches concurrent GUI workers in rounds and integrates their summaries. It provides evidence on decomposable interactive workloads, while identifying worker execution failures as a bottleneck. Its visual-history controls and multi-desktop setting make it unsuitable as a direct numerical baseline for this text-only experiment.³

2.2 Information transfer and evaluation

OrchBench represents dependencies and cross-agent transfers, then simulates quality, makespan and token cost. Its retention mechanism makes information loss a coordination variable. Our benchmark instead executes every worker against the API; simulated worker costs or outcomes are not substituted for observed usage or answers.⁴

CRAFT gives agents complementary partial views of a construction problem. It distinguishes individual reasoning from effective communication and describes repeated unproductive corrections. It motivates preserving source evidence and assessing the final result. Because every worker here receives the original task, we do not test its partial-observation problem.⁵

2.3 Skills and scoped workers

Subagents vs Agent Skills distinguishes procedural packages with explicit input/output interfaces from loosely structured skill text. Its main evaluation uses a selected 64-task subset and reports lower peak context with greater total token consumption. This supports testing payloads separately from topology.⁶

EvoAgent combines hierarchical delegation with keyword, embedding and model-based skill matching. Its domain-specific evaluation uses model judges; those scores do not establish exact execution reliability. We use its separation of retrieval and delegation as a reference, without implementing autonomous learning or reproducing its retrieval cascade.⁷

Swarm Skills packages roles, workflows, limits and dependencies as reusable coordination assets. Its qualitative case study leaves broad conformance testing unresolved. We treat portable workflow descriptions as a design proposal, not evidence that arbitrary multi-agent frameworks

¹Kulkarni and Kulkarni, arXiv:2603.22651v1, 2026.

²Wang et al., arXiv:2605.13172v1, 2026.

³Yu et al., arXiv:2607.22689v1, 2026.

⁴Ren et al., arXiv:2607.25656v1, 2026.

⁵Nath, VanderHoeven and Krishnaswamy, arXiv:2603.25268v2, 2026.

⁶Piriyakulkij et al., arXiv:2609.09233v1, 2026.

⁷A. Zhang et al., arXiv:2604.20133v3, 2026.

can interchange them without validation.⁸

Xu and Yan survey progressive disclosure: discover metadata, load chosen skill instructions, then retrieve supporting resources. This distinguishes a catalog from the content injected into each context. As a survey, it informs the loading policy but supplies no independent performance result for our architectures.⁹

2.4 Three additional references for adaptive coordination

Uno-Orchestra combines selective decomposition and delegation in a learned policy. It motivates deciding when delegation is worthwhile; we do not reproduce its training.¹⁰ ProgRouter guides execution decisions using progress, time and cost. Our gate is neither its learned predictor nor a router across different models.¹¹ TROVE retains valid continuations and edits routes in response to intermediate evidence. We use simple conditional review, without its offline-learned skill graph.¹² The expanded review comprises twelve arXiv papers, retaining identified versions and keeping their reported scores separate from this experiment.

3 Five architectures and the single-agent reference

All agents are stateless calls to the same model. A different role does not imply different trained weights. Handoffs contain visible answers, findings or unresolved issues, never private deliberation. Every call receives the full public task; partial observations and reduced production context are not evaluated.

Pattern	Implementation	Calls
Single	One solver returns the answer.	1
Sequential	Evidence extraction, solving and integration in three stages.	3
Parallel	Two complementary concurrent specialists followed by an integrator.	3
Hierarchical	Supervisor plans one or two jobs, concurrent workers execute, supervisor integrates.	3–4
Reflexive	Proposal and critique; revise if challenged, at most two rounds.	2–5
SAGE-CREH	Direct path or two solvers; review only upon defined signals.	1–3

3.1 SAGE-CREH: skill-aware gated execution

A lightweight deterministic coordinator routes using public task structure. There is no preliminary planning call. It selects one skill and chooses either one proposal or two complete concurrent solvers. The second solver prioritizes constraints, dependencies and tie breakers without seeing the first proposal. This adaptive redundancy differs from the fixed complementary division of work in the original parallel condition.

1. Identify the public contract and load only its relevant skill instructions.
2. Flag structural risk for a graph with at least eight nodes and any node with two or more

⁸X. Zhang et al., arXiv:2605.10052v2, 2026.

⁹Xu and Yan, arXiv:2602.12430v4, 2026.

¹⁰Cui et al., 2026.

¹¹Li, Abdelmoniem and Wang, 2026.

¹²T. Wang et al., 2026.

predecessors, or selection with at least ten items, forbidden pairs and prerequisites. These are fixed prior rules, not evaluation-learned thresholds.

3. Without risk, request one proposal. Finish if its schema is valid and it reports no specific unresolved issues; otherwise request a final review.
4. With risk, request two independent proposals concurrently. If both satisfy the contract, report no issues, and match exactly after JSON key sorting, finish without an integrator. Agreement does not establish correctness.
5. For disagreement, invalid shape or unresolved issues, an adjudicator checks candidates against the original data and returns the final answer.

The gate checks key sets, types and structure. It does not solve the task, execute SQL or inspect references or fixtures. An incorrect integer duration, for example, can pass this gate. Verbal confidence is not treated as a truth score. The adjudicator receives no evaluator feedback and can be wrong. There is no selective retry until a correct answer appears.

Let q be the fraction of tasks triggering two solvers, v the review rate on the direct path, and a the adjudication rate on the parallel path. With no transport failures, mean call count is

$$\mathbb{E}[N] = 1 + q + (1 - q)v + qa.$$

This equation describes measured mechanisms, not predicted accuracy. Parallel solver waiting time is approximately the maximum of their durations, not their sum; both solvers' tokens are still added. Avoiding an integrator removes one complete call, but is useful only if the result retains the required completion level.

3.2 Role assignment and skill distribution are different decisions

The adapter recognizes six closed-world interfaces from their fields rather than task IDs. It does not measure semantic skill-retrieval costs or errors in open-ended conversations.

Public signal	Loaded skill	Responsibility
Events with revisions	Ledger	Revision selection, status and monetary sums.
Participants	Calendar	Time zones and overlapping availability.
Task list	Dependencies	Precedence, earliest starts and critical path.
Candidate items	Selection	Budget, coverage and tie breakers.
Events with sequence and scope	Memory	Confirmations, deletions and inheritance.
SQL schema	SQL	Read-only queries and correct aggregation.

Workers for the same task need the same domain skill, even though their roles differ. We do not invent different skills per worker to suggest diversity. All ten original instruction bodies are reused unchanged. The four distractor skills are retained for ablations.

4 Experimental design

4.1 Separate development and freezing

Before replication, ten new instances were tested: two from each non-SQL family, with seeds distinct from the original tasks. The serial policy reviewed risky tasks after a proposal; the parallel

policy used two simultaneous solvers. Selection favored most strict successes, breaking ties by fewer tokens and then less time. SQL contracts were tested offline; there was no new live SQL pilot. Both candidates' results and resource use are reported separately.

The protocol is local and precedes execution; it is not an independent preregistration. Code, prompts, limits and order were copied and hashed before each phase. A Python-tuple versus JSON-list comparison correction was made before the first evaluation call, without changing tasks or policy; its amendment is retained. The architecture did not change during final replication.

4.2 Tasks and paired comparison

The exact twelve original problems and their fixtures were repeated twice per condition. Each of six families contains two problems: ledgers with 18 events, calendars with three participants, eleven-node graphs, selecting four out of twelve items, memory with 24 events, and two SQL queries. Ten non-SQL tasks require exact matching; each query runs against three test databases hidden from agents, with read-only permissions.

Nine conditions were run: single agent; the four original architectures; SAGE-CREH; hierarchy with ten skills; SAGE without delegation; and SAGE with ten skills. This yields 216 runs, 24 per condition. Task-repetition blocks and then conditions within each block were randomized with seed 20260914. Conditions did not execute concurrently; at most two calls could overlap within one flow.

4.3 Controls, limits and potential confounds

GPT-5.6 Luna, low reasoning effort, Responses API, JSON-object, non-streaming generation and a maximum of 1536 output tokens per call, including reasoning, were held fixed. Connection and read timeouts were 10 and 60 seconds; no transport retries; at most five calls overall and a 240-second deadline for starting another call. That deadline does not interrupt an HTTP request already underway.

Original controls retain their prompts and ten-description catalog plus a skill body selected by family. SAGE omits the catalog and explicitly states JSON nesting, especially for SQL. Comparisons therefore change more than topology. Two controls help isolate effects: *SAGE without delegation* retains its proposer and payload but always makes one call; *SAGE with ten skills* retains policy and prompts while loading every body. The original hierarchy pair is also rerun. Maximum per-call budgets are identical, not the actual tokens consumed.

4.4 Measurement and analysis

Primary completion requires every field to be correct in the original JSON envelope. A predefined secondary v2 metric wraps a raw SQL string under **answer** into the nested **sql** field and applies the same evaluator. No SQL text is edited and no non-SQL answer is corrected. Primary scores are never repaired.

Tokens sum provider-reported input and output across all calls, including supervision, critique, repeated context and observed failures. Cached input and reasoning are already part of those totals. Monotonic wall time includes coordination, network and traces through the final response, excluding grading. It is neither summed overlapping HTTP durations nor a throughput benchmark.

Exploratory 95% percentile intervals use 10000 paired task-cluster bootstrap draws, keeping both

repetitions and all conditions together. Completion, token and time differences and token ratios are reported. Twelve distinct tasks are not 216 independent tasks. Degenerate intervals at a ceiling do not establish perfect reliability.

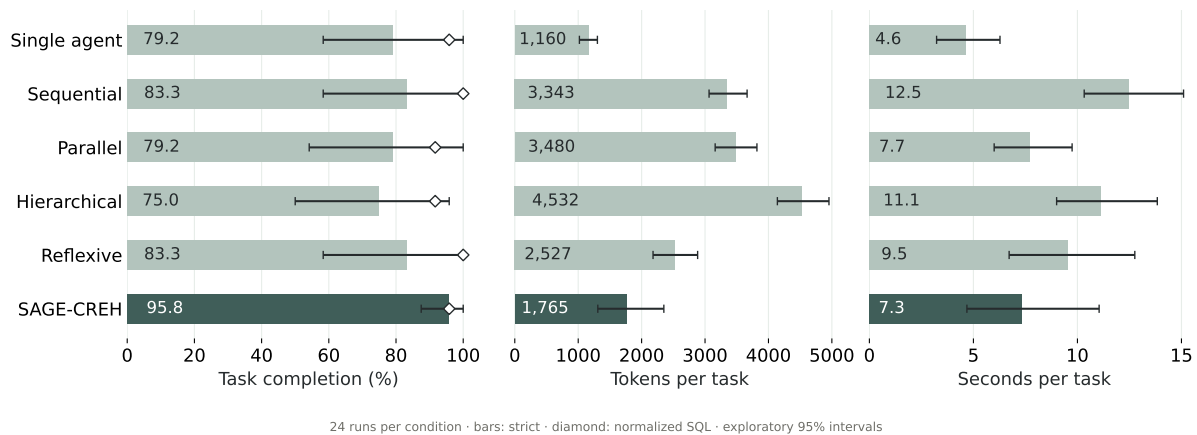
5 Version-2 replication results

The 216 runs completed 510 calls and 707,090 tokens, with 0 infrastructure or protocol errors. Call accounting was complete; tokens were not estimated.

5.1 Main comparison: same tasks and model

Configuration	Strict	SQL norm.	Tokens	s/task	Calls
Single agent	19/24	23/24	1,160	4.62	24
Sequential	20/24	24/24	3,343	12.47	72
Parallel	19/24	22/24	3,480	7.69	72
Hierarchical	18/24	22/24	4,532	11.13	96
Reflexive	20/24	24/24	2,527	9.54	54
SAGE-CREH	23/24	23/24	1,765	7.34	38

Each row contains 24 runs. Tokens and seconds are per-task means; calls are the condition total. SQL norm. is a secondary format metric, not answer repair.



Strict-completion bars and normalized-SQL diamonds. Task-cluster 95% percentile intervals do not establish a general architecture ranking.

5.2 Completion by task family

Family	Single	Seq.	Par.	Hier.	Refl.	SAGE
Ledger	4 / 4	4 / 4	4 / 4	4 / 4	4 / 4	3 / 3
Calendar	3 / 3	4 / 4	4 / 4	2 / 2	4 / 4	4 / 4
Dependencies	4 / 4	4 / 4	2 / 2	4 / 4	4 / 4	4 / 4
Selection	4 / 4	4 / 4	4 / 4	4 / 4	4 / 4	4 / 4
Memory	4 / 4	4 / 4	4 / 4	4 / 4	4 / 4	4 / 4
SQL	0 / 4	0 / 4	1 / 4	0 / 4	0 / 4	4 / 4

Each cell: strict successes / normalized-SQL successes, both out of four family runs (two tasks times two repetitions). Outside SQL the metrics are identical.

5.3 Skill-loading and delegation controls

Configuration	Strict	SQL norm.	Tokens	s/task	Calls
Hierarchical	18/24	22/24	4,532	11.13	96
Hierarchy + 10 skills	20/24	22/24	8,578	10.62	96
SAGE-CREH	23/24	23/24	1,765	7.34	38
SAGE without delegation	21/24	21/24	1,110	5.93	24
SAGE + 10 skills	22/24	22/24	2,966	6.38	34

SAGE without delegation retains its proposer and instructions, removing only the second solver and review. SAGE with ten skills retains the policy and changes instruction loading. Original controls include a description catalog absent from SAGE; do not attribute every difference to topology.

5.4 Paired token and time differences

Comparison	Token change (%)	95% CI	Seconds difference	95% CI
Single agent	+52.2	[15.9, 95.3]	+2.73	[1.20, 4.97]
Sequential	-47.2	[-61.5, -29.5]	-5.13	[-6.29, -3.77]
Parallel	-49.3	[-62.3, -33.8]	-0.35	[-1.73, 1.52]
Hierarchical	-61.1	[-71.4, -48.5]	-3.79	[-5.25, -2.30]
Reflexive	-30.2	[-51.8, -3.5]	-2.20	[-5.33, 0.50]
SAGE without delegation	+59.1	[21.3, 99.0]	+1.42	[0.46, 2.58]
SAGE + 10 skills	-40.5	[-49.7, -29.6]	+0.96	[-0.49, 2.94]

Each row is SAGE minus the indicated configuration; token percentage change is $100(\text{SAGE}/\text{control} - 1)$. Negative values favor lower SAGE consumption or time. Exploratory intervals use 10000 paired task-cluster draws.

5.5 Executed routes and revision outcomes

SAGE-CREH. Direct: 13; Parallel agreement: 5; Parallel adjudication: 3; Selective review: 3. Relative to the first candidate: 5 corrections and 0 regressions in strict completion.

SAGE without delegation. Direct: 24. Relative to the first candidate: 0 corrections and 0 regressions in strict completion.

SAGE + 10 skills. Direct: 15; Parallel agreement: 7; Parallel adjudication: 1; Selective review: 1. Relative to the first candidate: 2 corrections and 0 regressions in strict completion.

This inspection uses the evaluator after execution and never fed the routing policy. A correction may concern formatting. Per-run details are retained.

5.6 Development: excluded from replication

Serial: 9/10; 1,606.7 tokens and 9.07 mean seconds.

Parallel: 10/10; 1,628.4 tokens and 8.11 mean seconds.

The parallel policy was selected for higher completion under the prior rule. Development used 32,351 tokens in 28 calls and 20 runs. This cost is reported separately and excluded from replication means.

The sole success separating candidates occurred on calendar task DEV22: both policies used the same direct path, without risk or review. Generation variability can explain this difference; it does not establish a causal advantage of parallelism. Selection is retained under the prior rule, not reinterpreted as proof of superiority.

6 Interpretation and application

SAGE-CREH achieved the highest strict completion in the main comparison: 23/24 (95.8%), averaging 1765 tokens and 7.34 seconds. Relative to the four original patterns, tokens fell by 47.2% versus sequential, 49.3% versus parallel, 61.1% versus hierarchical and 30.2% versus reflexive. These are observed differences; strict-completion difference intervals include zero. Its mean time advantage over parallel was only 0.35 seconds, with an interval including zero; a latency advantage over reflexive is likewise not established.

The conclusion depends on the output contract. With normalized SQL, sequential and reflexive reached 24/24; SAGE and the original single agent reached 23/24. The latter averaged 1160 tokens and 4.62 seconds, less than SAGE. Among the six main designs, the point-estimate Pareto frontier contains single agent and SAGE under strict scoring, and single agent and reflexive under secondary scoring. These are estimated frontiers, not population-dominance tests. SAGE is therefore not the best choice under every definition of success.

SAGE without delegation reached 21/24 with 1110 tokens: adding coordination raised consumption by 59.1% and yielded two additional successes in the sample, without a conclusive completion difference. Its two SQL failures contain an extra `uncertainties` key in the final envelope. That form is not covered by the predefined secondary normalization, which only wraps raw SQL strings. Thus even the secondary column does not separate every possible formatting error from content correctness.

Inspection of SAGE's own traces identifies five improvements over its first candidate: three SQL-contract reviews and two adjudications involving dependencies and selection. No strict regressions followed review. Thirteen direct responses and five parallel agreements finished without an integrator; three adjudications and three selective reviews yielded 38 calls over 24 tasks. These are concrete mechanisms, not a reason to label every contract correction superior multiagent reasoning.

SAGE's only final failure occurred on T02, first repetition: all monetary totals matched, but it returned seven events instead of eight. The contract was valid, no issues were reported and the direct path accepted it. This case identifies precisely what a shape-only gate does not detect.

Selecting instructions reduced tokens by 40.5% versus SAGE with ten skills (1765 versus 2966). Observed time did not fall: 7.34 versus 6.38 seconds, with an uncertain difference. Responses also changed routes: 38 versus 34 calls. This ablation measures the total loading effect under the same policy, not an identical executed graph. Both hierarchy conditions made 96 calls; loading ten skills raised tokens by 89.3% without increasing normalized successes (22/24 in both).

For this strict contract, SAGE is a lower-consumption adaptive alternative to the four fixed teams. If the system accepts SQL-envelope normalization and prioritizes cost, the evidence favors the

single-agent reference. If observed normalized completion is the priority, it favors reflexive over sequential for lower mean consumption and time. This conditional choice reflects the evidence more faithfully than proclaiming a universally winning architecture. The evaluated proposal retains a path without subagents and uses redundancy only for part of the workflow. It does not include browsing, terminal interaction, external tools, persistent memory, write coordination or production permissions. A deployed implementation must replace or extend the contract adapter with skill retrieval, retrieval-error measurement and role-specific authority limits.

Architecture choice should separate solver sufficiency, risk detection, instruction delivery and transfer costs. A fixed integrator consumes resources even when workers agree; early exit can avoid that step, but can also accept a shared error. More roles or different skill labels do not create statistical independence between responses from the same model.

7 Validity and reproducibility

Final tasks were already known when designing SAGE: this is a development-informed paired replication, not a blind generalization test. Fresh development seeds do not remove knowledge of task families or prior failures. The sample is small and synthetic, with one model, two repetitions and short contexts; ceiling effects and provider/network fluctuations remain. RAM, complete-project success and equal budgets per correct response are not measured.

The package contains both waves separately, development attempts, policy decision, frozen code, protocols, tasks, evaluator references, visible per-call traces, CSV, XLSX and bilingual LaTeX sources. It contains no credentials or private deliberation. Credentials were loaded from an authorized local file outside the public package. Historical measurements do not substitute for replication controls and are not pooled with them to estimate latency.

Appendix: historical v1 results

The earlier wave contained 144 runs, 416 calls and 568,283 tokens, preserved unchanged. SQL normalization was post hoc in v1 and predefined as secondary in v2. Waves are not pooled.

Configuration	Strict	SQL norm.	Tokens	s/task	Calls
Single agent	20/24	23/24	1,143	4.72	24
Sequential	20/24	24/24	3,410	13.72	72
Parallel	19/24	23/24	3,520	8.14	72
Hierarchical	20/24	24/24	4,373	11.19	95
Reflexive	20/24	24/24	2,714	9.77	58
Hierarchy + 10 skills	21/24	24/24	8,518	11.93	95

Reproducing and auditing the materials

The public archive contains code, data and editable sources for both versions. Extract it and work from its root, installing `requirements.txt` dependencies in an isolated environment. To verify and regenerate the preserved wave without calling the API:

```
python3 -m unittest discover -s benchmark -v
python3 -m unittest discover -s extension -v
python3 extension/analyze.py
python3 extension/write_results.py
python3 extension/write_web.py
```

Offline tests do not produce model scores. The analyzer checks all 216 runs, task identity, order, frozen copies, loaded instructions and per-call usage. The generator writes numeric sections; authored conclusions remain separate. Compile each `article.tex` twice from `paper-v2/es` and `paper-v2/en` using pdfLaTeX.

A new live replication requires an authorized key and incurs API cost. Always use a new output directory and follow `extension/README.md`; do not overwrite the policy decision or published traces. The model is an alias, so later replication does not guarantee identical answers or time.

Package v2 integrity can be checked with `ARTIFACTS-v2.sha256`; downloadable files additionally have a `SHA256SUMS.txt` manifest. Traces contain synthetic public inputs, visible responses and reported usage, never authorization headers.

References

1. Siddhant Kulkarni and Yukta Kulkarni. *Benchmarking Multi-Agent LLM Architectures for Financial Document Processing: A Comparative Study of Orchestration Patterns, Cost-Accuracy Tradeoffs and Production Scaling Strategies*. 2603.22651v1. First submitted 2026-03-24. <https://arxiv.org/html/2603.22651v1>
2. Ziqi Wang, Yuhao Yang, Zhiwei Ling, Wenzhuo Qian and Hailiang Zhao. *When Does Hierarchy Help? Benchmarking Agent Coordination in Event-Driven Industrial Scheduling*. 2605.13172v1. First submitted 2026-05-13. <https://arxiv.org/html/2605.13172v1>
3. Zedong Yu et al.. *Beyond Sequential Interaction: Benchmarking Parallel Execution and Coordination for GUI Agents*. 2607.22689v1. First submitted 2026-07-17. <https://arxiv.org/html/2607.22689v1>
4. Zhenzhen Ren et al.. *OrchBench: Evaluating Multi-Agent Orchestration Plans in Isolation via Deterministic Simulation*. 2607.25656v1. First submitted 2026-07-28. <https://arxiv.org/html/2607.25656v1>
5. Abhijnan Nath, Hannah VanderHoeven and Nikhil Krishnaswamy. *CRAFT: Grounded Multi-Agent Coordination Under Partial Information*. 2603.25268v2. First submitted 2026-03-26. <https://arxiv.org/html/2603.25268v2>
6. Wasu Top Piriyaakulkij, Rachel Lawrence, Alicia Curth, Sushrut Karmalkar and Niranjani Prasad. *Sub-agents vs Agent Skills: Executing Reusable Knowledge for Long-Horizon Agentic Tasks*. 2609.09233v1. First submitted 2026-09-07. <https://arxiv.org/html/2609.09233v1>
7. Aimin Zhang, Jiajing Guo, Fuwei Jia, Chen Lv, Boyu Wang and Fangzheng Li. *EvoAgent: An Evolvable Agent Framework with Skill Learning and Multi-Agent Delegation*. 2604.20133v3. First submitted 2026-04-22. <https://arxiv.org/html/2604.20133v3>
8. Xinyu Zhang et al.. *Swarm Skills: A Portable, Self-Evolving Multi-Agent System Specification for Coordination Engineering*. 2605.10052v2. First submitted 2026-05-11. <https://arxiv.org/html/2605.10052v2>
9. Renjun Xu and Yang Yan. *Agent Skills for Large Language Models: Architecture, Acquisition, Security, and the Path Forward*. 2602.12430v4. First submitted 2026-02-12. <https://arxiv.org/html/2602.12430v4>
10. OpenAI. *GPT-5.6 Luna Model*. Accessed 2026-09-13. <https://developers.openai.com/api/docs/models/gpt-5.6-luna>
11. Zhiqing Cui et al. *Uno-Orchestra: Parsimonious Agent Routing via Selective Delegation*. 2605.05007v1. First submitted 2026-05-06. <https://arxiv.org/html/2605.05007v1>
12. Songyuan Li, Ahmed M. Abdelmoniem and Shiqiang Wang. *ProgRouter: Online Progress-Guided Orchestration for Multi-Agent LLM Workflows under Quality-Cost Tradeoffs*. 2608.25992v1. First submitted 2026-08-26. <https://arxiv.org/html/2608.25992v1>
13. Tianxing Wang et al. *TROVE: Adaptive Agent Skill Orchestration via Trace-Grounded Route Validation and Editing*. 2609.05019v1. First submitted 2026-09-04. <https://arxiv.org/html/2609.05019v1>